STUDENT RECORD AND ACADEMIC MANAGEMENT SYSTEM

ALGORITHMS • FLOWCHARTS • PSEUDOCODE • TESTING

Student Name: Srushti Bhausaheb Kank

PRN: 2126UDSF1006

Course: FY BTech – Artificial Intelligence and Data Science

Subject: Introduction to Programming and Data Structures

1. Problem Statement

Manual student record keeping can make searching and academic calculations time-consuming. This project organizes student IDs, names, departments and marks and offers common record-management operations through a console menu.

2. Objectives

• Add, search, update and delete student records.
• Display records, calculate average marks and find the highest scorer.
• Filter students by department and list unique departments.
• Demonstrate strings, lists, tuples, sets and dictionaries.

3. Data Structures Used

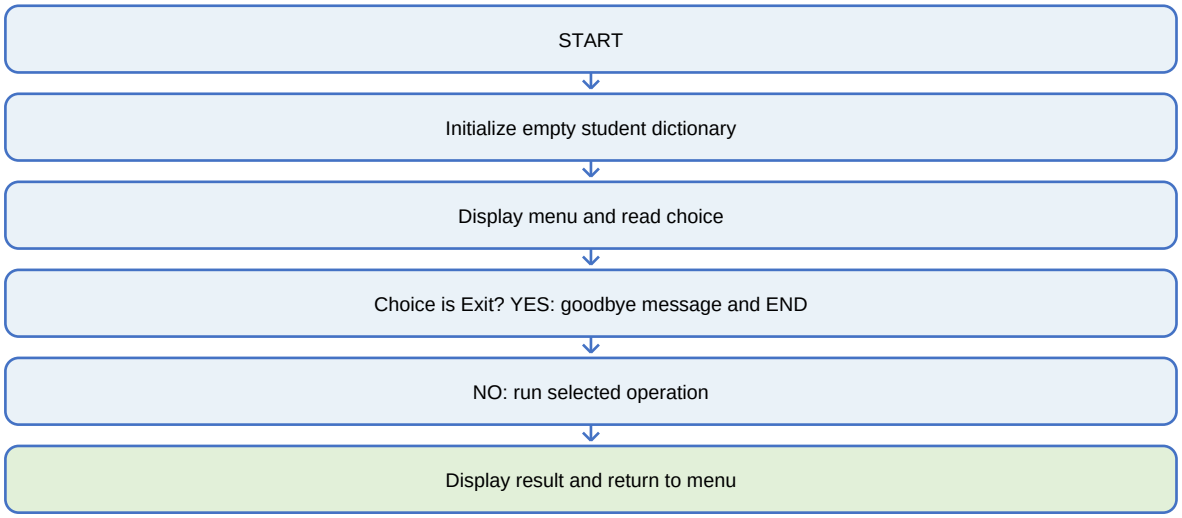
Structure	Purpose
String	Student ID, name, department and menu choice.
Dictionary	Maps each student ID to its record.
Tuple	Stores name, department and marks as one record.
List	Collects matching records during department filtering.
Set	Stores unique department names.

4. Main Algorithm

1. Start and initialize an empty student dictionary.
2. Display the menu and read the user's choice.
3. Execute the selected operation.
4. Validate input and display the result or an error.
5. Repeat until Exit is selected.
6. End.

5. Main Program Flowchart

Main Menu

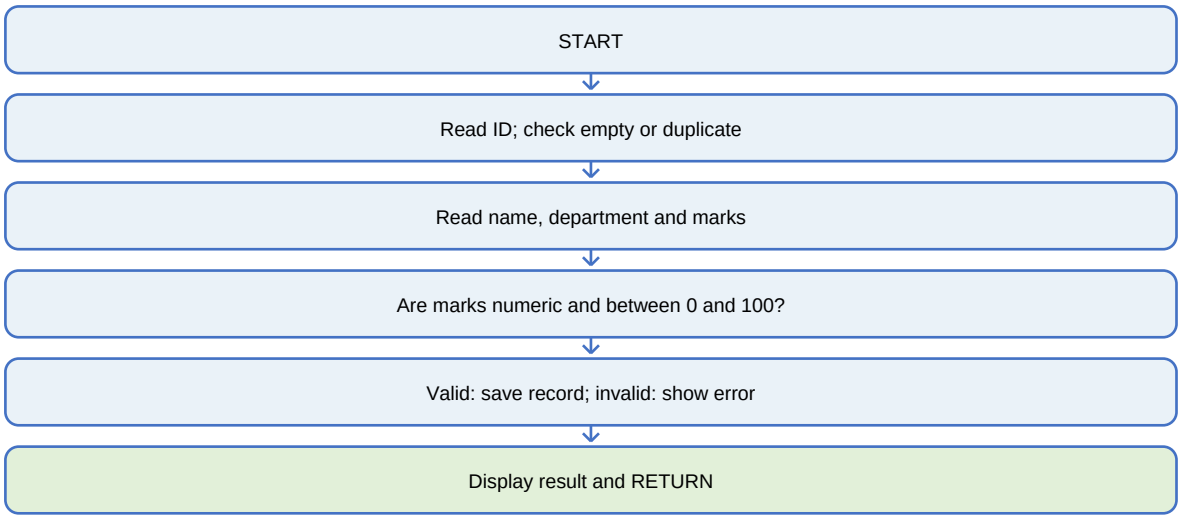


6. Operation Algorithms and Flowcharts

A. Add Student

Read ID; reject empty or duplicate IDs. Read name, department and marks. Validate marks from 0 to 100. Store the tuple in the dictionary and confirm.

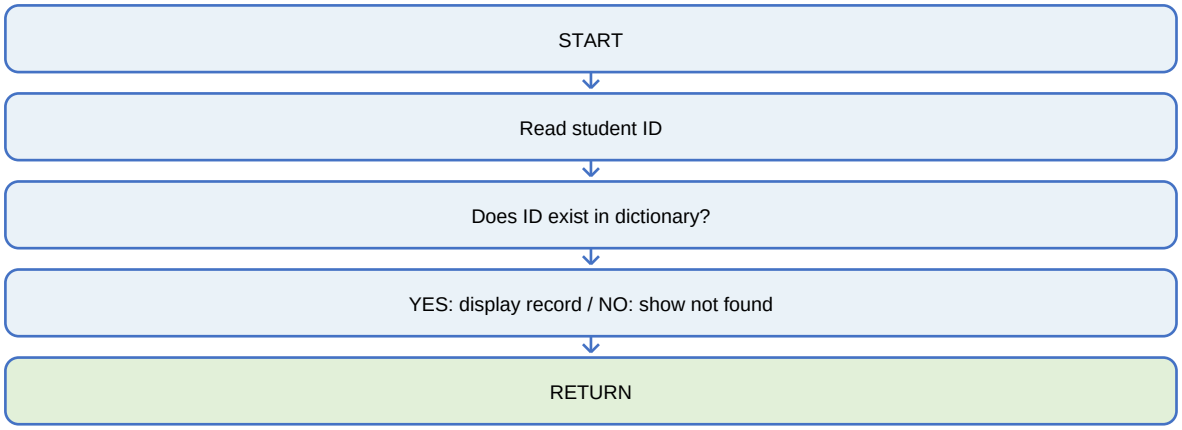
Add Student



B. Search Student

Read ID. If it exists, display the record; otherwise display 'Student not found'. Return to menu.

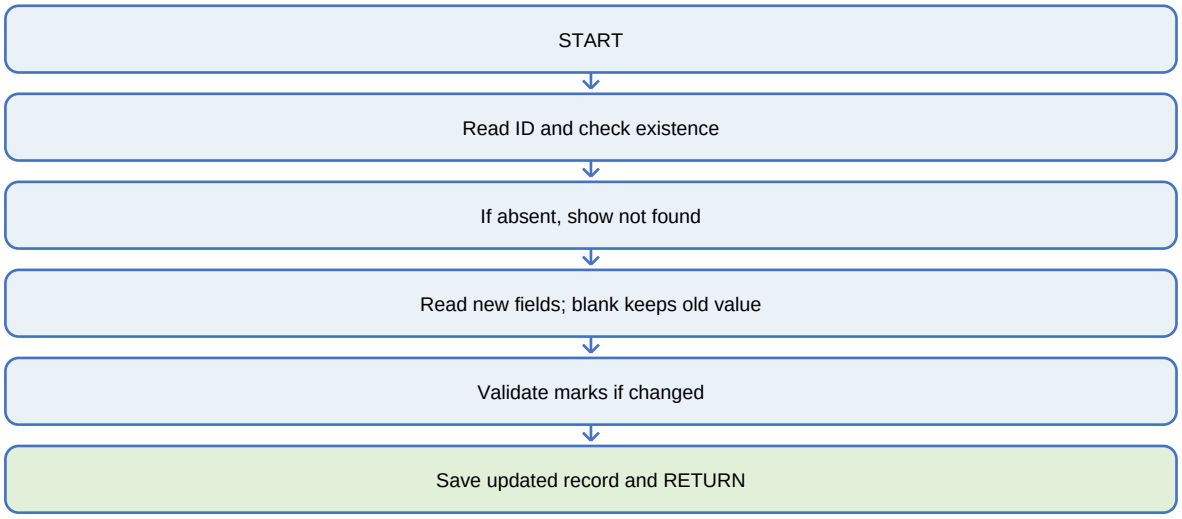
Search Student



C. Update Student

Read ID and check existence. Ask for updated fields; blank input keeps the old value. Validate new marks, save the updated tuple and confirm.

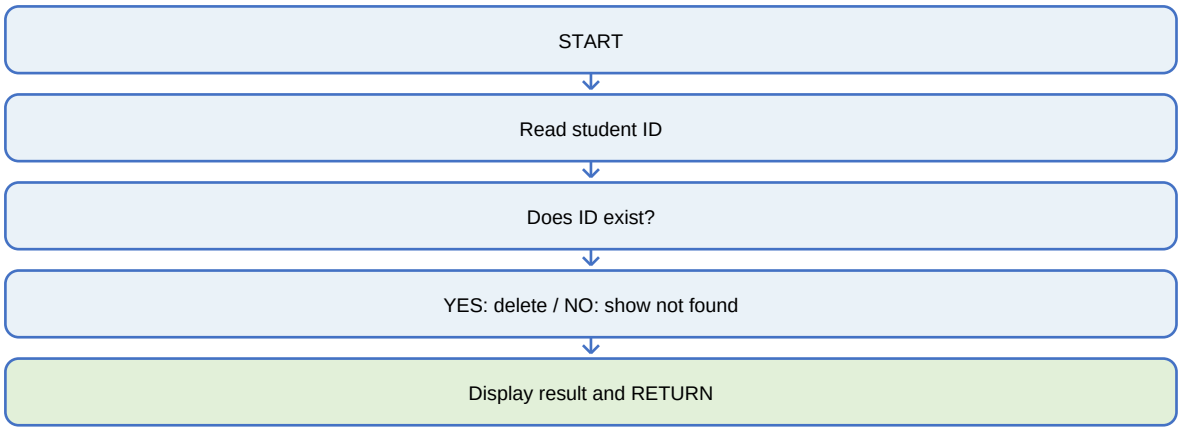
Update Student



D. Delete Student

Read ID. If it exists, delete the dictionary entry and confirm; otherwise show not found.

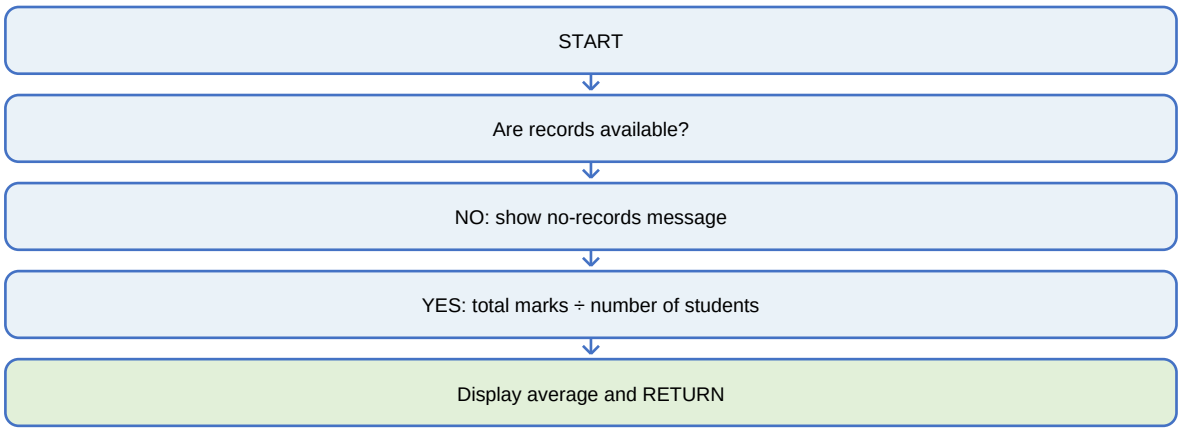
Delete Student



E. Calculate Average Marks

If no records exist, show a message. Otherwise sum marks, divide by number of records and display average to two decimal places.

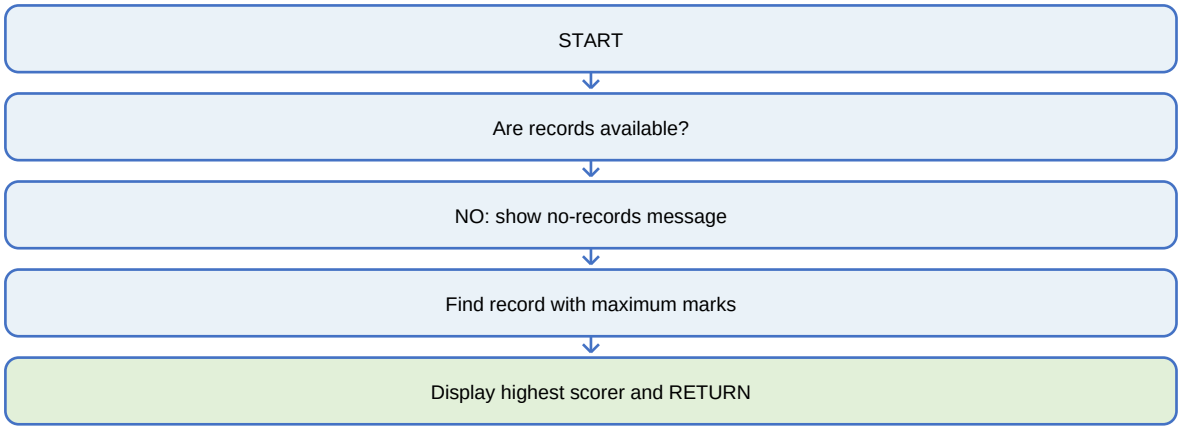
Average Marks



F. Find Highest Scorer

Check that records exist. Find the record with the greatest marks and display the student's name, ID and marks.

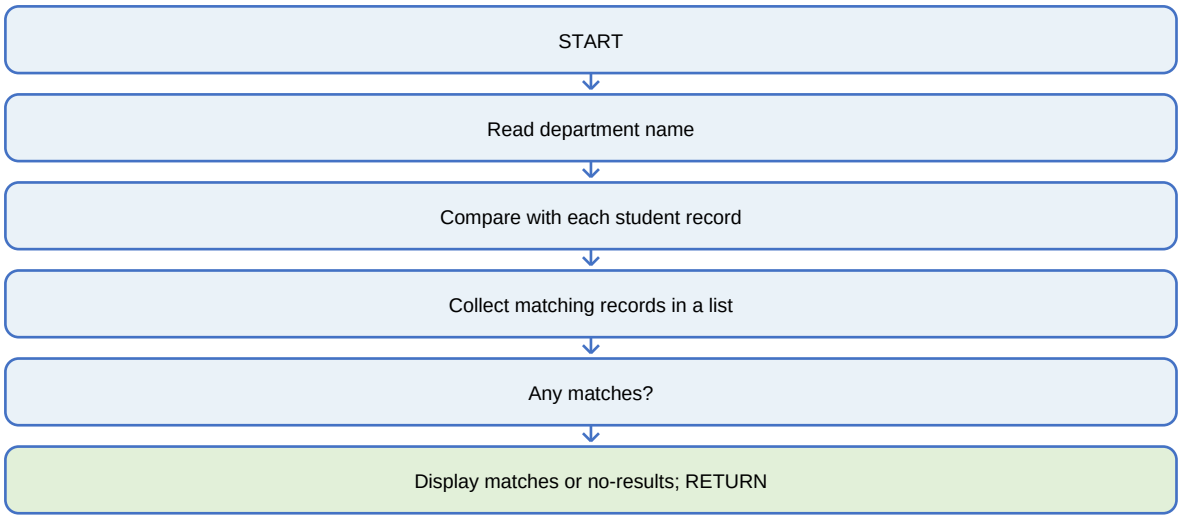
Highest Scorer



G. Filter by Department

Read department name. Compare case-insensitively with each student's department, collect matches in a list, then display matches or a no-results message.

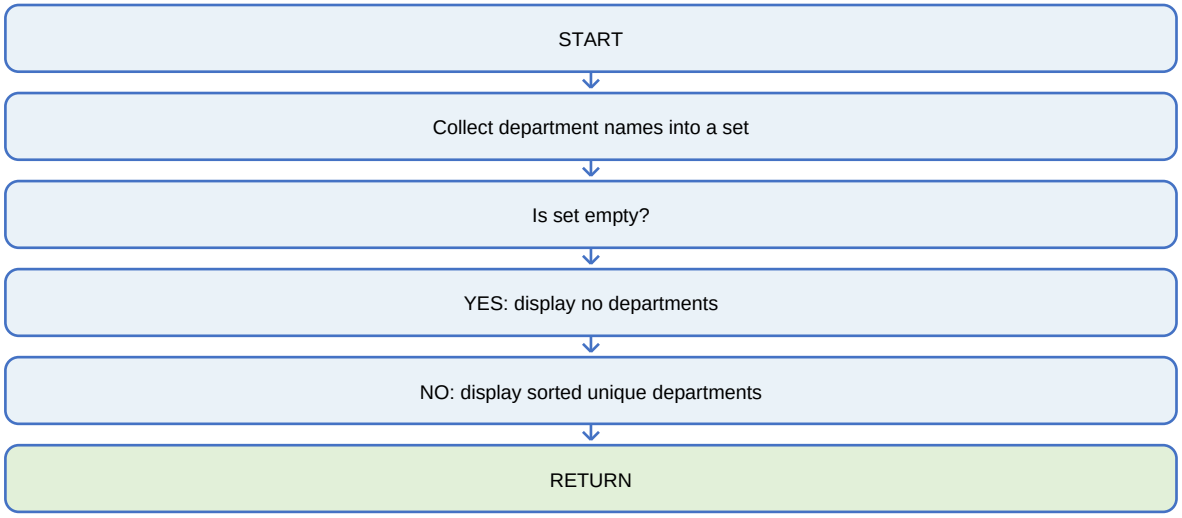
Department Filter



H. Show Unique Departments

Build a set from department names. If empty, display a message; otherwise display sorted unique department names.

Unique Departments



7. Pseudocode

```
BEGIN
    students <- empty dictionary
    REPEAT
        DISPLAY menu
        INPUT choice
        IF choice = 1: add student after validation
        ELSE IF choice = 2: search by student ID
        ELSE IF choice = 3: update existing record
        ELSE IF choice = 4: delete existing record
        ELSE IF choice = 5: display all records
        ELSE IF choice = 6: calculate average marks
        ELSE IF choice = 7: display maximum-mark student
        ELSE IF choice = 8: filter records by department
        ELSE IF choice = 9: display unique departments
        ELSE IF choice = 0: display goodbye message
        ELSE: display invalid choice
    UNTIL choice = 0
END
```

8. Testing Table

Test case	Input / condition	Expected result
Valid add	ID S01, Asha, AIDS, 85	Record saved
Duplicate ID	Add S01 again	Duplicate warning
Search existing	Search S01	Record displayed
Search missing	Search S99	Student not found
Invalid marks	121 or text	Validation error
Average	Marks 85 and 75	Average 80.00
No records	Average before adding	No-records message

9. Limitations and Future Scope

Records are held in memory and disappear when the program closes. Future versions could save data to a file or database, add login security, support multiple subjects and generate reports.

10. Conclusion

This project demonstrates how Python collections and functions organize student information, validate input and perform common academic record operations.

11. GitHub README Summary

Project: Student Record and Academic Management System
Language: Python 3
Features: Add, search, update, delete, display, average marks, highest scorer, department filter and unique departments.
Run: Save as `student_record_system.py` and execute with Python 3.